التقرير الفني الشامل للمشروع

منصة CodeTree V2 — التوثيق المعماري وآلية العمل الكاملة

شرح تفصيلي دقيق لكل مكونات وتدفقات البيانات داخل تطبيق **CodeTree V2**، وهو منصة برمجية تعليمية هجينة تعتمد على معمارية الخدمات المصغرة (Polyglot Microservices) لتقييم وتصحيح الأكواد آلياً (Moulinette) مع نظام تلعيب تفاعلي (Gamification).

المعمارية:	اللغات والتقنيات:	إصدار التوثيق:	تاريخ الإصدار:
Microservices (Polyglot)	**Java, C# .NET, Python, React**	**V2.0 (Official Architecture)**	**2026**

1 نظرة عامة على التطبيق وفلسفة بنائه

تطبيق **CodeTree V2** هو منصة متقدمة مصممة لتجربة تعليم البرمجة الممارسة العملية (Coding by Doing)، وهي مستوحاة من منصات عالمية رائدة مثل *LeetCode* و *HackerRank* ونظام التقييم الآلي الصارم **Moulinette** المعتمد في منظومة مدارس 42 و 1337 البرمجية.

لماذا اختيرت معمارية Polyglot Microservices؟

بدلاً من بناء تطبيق موحد (Monolithic)، تم تقسيم النظام إلى خدمات مستقلة لكل منها دور متخصص تماماً، مع استخدام أفضل أداة لكل مهمة: الربط والشبكات عبر **Spring Cloud** لقوتها واستقرارها المؤسسي، وإدارة المستخدمين والمحتوى عبر **.NET 8** لسرعة معالجة الـ APIs وقوة Entity Framework، ومحرك تشغيل الأكواد عبر **Python / FastAPI** لقدرتها العالية على إدارة العمليات البرمجية الخارجية (Subprocesses) ومطابقة النصوص والتصحيح الآلي.

2 خريطة المنظومة وتدفق البيانات (System Architecture & Routing)



3 تشريح الخدمات المصغرة وآلية عمل كل خدمة بالتفصيل

1. خادم الاكتشاف (Discovery Service) 8761:

Spring Cloud Netflix Eureka

الدور: سجل الهاتف المركزي (Service Registry) للنظام بالكامل.

- تتصل به كل خدمة تلقائياً عند إقلاعها وتُعلن عن اسمها ومكان وجودها (IP + Port).
- يستقبل نبضات دورية (Heartbeats)؛ إذا انقطعت إشارة أي خدمة تُحذف من السجل فوراً لمنع سقوط الطلبات.
- يسمح لـ Gateway بتوجيه الطلبات عبر الأسماء البرمجية (مثل `lb://CONTENT-SERVICE`) دون الحاجة لـ Hardcoded IPs.

2. بوابة الواجهات (API Gateway) 8080:

Spring Cloud Gateway

الدور: البوابة الحصرية والمحصنة لدخول ومغادرة جميع طلبات الواجهة.

- **إدارة الـ CORS:** مجهزة للسماح فقط لتطبيق الـ React `http://localhost:5173` بكافة طرق HTTP.
- **المطابقة المسارية (Route Matching):** فرز المسارات الموجهة إلى `/api/content`، `/api/auth`، `/api/gamification`.
- **توزيع الحمل (Load Balancing):** موازنة الاستدعاءات ديناميكياً بين أي عدد من نُسخ الخدمات المشغلة.

3. خدمة الهوية (Identity Service) 5026:

NET 8 Web API + JWT.

الدور: إدارة حسابات المستخدمين والطلاب والمعلمين وتأمين النظام.

- **التسجيل (Register):** إنشاء حساب مشفر بالكامل بالاعتماد على `ASP.NET Identity` في قاعدة بيانات `codetree_identity`.
- **المصادقة (Login):** إصدار توكن JWT موقّع بمفتاح سري (HMAC-SHA256) يحمل الـ Claims (UserId, Roles, Email) صالح لـ 3 ساعات.
- **Steeltoe Discovery:** تسجيل ذاتي سلس في خادم Eureka المكتوب بلغة جافا.

4. خدمة المحتوى (Content Service) 5229:

NET 8 + EF Core.

الدور: إدارة الكورسات، المسارات البرمجية، بنك المسائل والتمارين.

- **نموذج Course:** يحتوي على معرف الدورة، العنوان، الوصف، واللغة المستهدفة (C, Python, C#, Java).
- **نموذج Lesson:** الدروس والتمارين المرتبطة بالكورس وترتيبها ومحتواها التفصيلي.
- **مخرجات التقييم:** يخزن المعلم نص المسألة متضمناً علامة `Output: [Expected]` لتقوم Moulinette باختباره تلقائياً.

5. خدمة التحفيز (Gamification) 8082:

Spring Boot 3 + Spring Data JPA

الدور: مكافأة الطلاب بالنقاط والمستويات ولوحة المتصدرين.

- **سجل التقدم (UserProgress):** ربط كل `userId` بنقاط الخبرة (XP) والمستوى (Level).
- **معادلة الترقي الذاتية:** `Level = (Points / 100) + 1` — كل 100 نقطة تمنح الطالب ترقية فورية.
- **لوحة الشرف (Leaderboard):** تزويد لوحات التحكم بالترتيب التنازلي للطلاب الأعلى إنجازاً.

6. محرك الأكواد (Code Engine / Moulinette) 8000:

Python 3.10 + FastAPI

الدور: تصحيح وتنفيذ الأكواد في بيئة معزولة ومحمية.

- **تنفيذ Python:** تشغيل في `subprocess` مستقل مع سقف زمني إجباري (5 ثوانٍ) لمنع التجميد.
- **مترجم لغة C الكامل:** كتابة الكود داخل مجلد معزول `tempfile`، ثم التجميع عبر `gcc` والتشغيل والتقاط مخرجات `stdout` و `stderr`، ثم إبادة الملفات بعد التنفيذ.

4 هندسة الواجهة الأمامية وشاشات تجربة المستخدم (Frontend UX)

تم بناء الواجهة باستخدام **React 19 و Vite و TailwindCSS 4** لتقديم تجربة فائقة السرعة مع تصميم عصري بنمط داكن (Dark Theme) مستوحى من بيئات المطورين الاحترافية:

الصفحة	المسار	الخدمة المرتبطة	الوظيفة والآلية
الرئيسية (Dashboard)	`/`	Content + Gamification	عرض الدورات البرمجية، وبطاقة مستوى الطالب وإجمالي نقاط XP المحصلة.
محرر الأكواد (Code Engine)	`engine/`	Code Engine + Gamification	محرر كامل مقسم مع نافذة طرفية سوداء (Terminal) لتشغيل الكود وتقييم حل التمرين تلقائياً ومكافأة الطالب بالنقاط.
تفاصيل الدورة (Course Details)	`course/:id/`	Content Service	قائمة التمارين والدروس المرتبطة بالكورس، مع زر مباشر لحل كل تمرين في المحرر.
لوحة المعلم (Teacher Dashboard)	`teacher/`	Content + Gamification	إنشاء دورات جديدة، كتابة تمارين وإرفاق نص المخرج المطلوب، ومتابعة لوحة صدارة الطلاب.
لوحة الإدارة (Admin Panel)	`admin/`	Identity + Services Health	فحص نبض وحالة السيرفرات الستة، وإدارة حسابات الطلاب والمعلمين وإمكانية حظرهم.
الاشتراكات (Pricing)	`pricing/`	مستقلة / بوابة دفع	عرض باقات الاشتراك للطلاب والمدارس والمؤسسات مع نموذج دفع بنكي مشفر.
تسجيل الدخول والتسجيل	`login &/ /register`	Identity Service	نماذج الدخول وتخزين التوكن و UserId في `localStorage`.

5 دورة التقييم الآلي للأكواد البرمجية (The Moulinette Test Cycle)

كيف تتحقق المنصة من صحة كود الطالب وتمنحه النقاط دون أي تدخل بشري؟

خطوات دورة التقييم بالتسلسل:

1. **استخراج النتيجة المتوقعة:** يقرأ المحرر نص التمرين المخزن في قاعدة البيانات، ويبحث عبر التعبير النمطي عن السطر التالي: `Output:\s*(.*)`.
2. **التنفيذ المعزول:** يُرسل كود الطالب (سواء Python أو C) إلى `http://localhost:8000/execute`. يقوم المحرك بترجمته وتشغيله داخل Sandbox ويعيد النص المطبوع الفعلي.
3. **المطابقة الذكية:** يقارن النظام النتيجة الفعلية مع المتوقعة بعد تنظيف الفراغات (Trim).
4. **الاعتماد والمكافأة:** إذا تطابقت النتائج، يطبع المحرر ✅ `TEST PASSED! Output matches expected result`، ثم يرسل طلباً خلفياً:

```
POST http://localhost:8080/api/gamification/progress/{userId}/add-points?points=10
```

تستقبل خدمة التحفيز الطلب، وتضيف 10 نقاط للطالب وتحدث مستواه في قاعدة البيانات فوراً.

6 معمارية قواعد البيانات وإعدادات Docker

تطبيقاً لمبدأ الخدمات المصغرة **"قاعدة بيانات لكل خدمة" (Database per Service)**، تم فصل البيانات لمنع التداخل والاعتمادية المتبادلة:

قاعدة البيانات	الخدمة المالكة	الجداول الرئيسية	الوصف
`codetree_identity`	Identity Service	`AspNetUsers`, `AspNetRoles`, `AspNetUserClaims`	تخزين بيانات المستخدمين، التجزئة المشفرة لكلمات السر، والأدوار والصلاحيات.
`codetree_content`	Content Service	`Courses`, `Lessons`	تخزين الدورات ولغات البرمجة ونصوص التمارين ونواتج التصحيح المطلوبة.
`codetree_gamification`	Gamification Service	`UserProgress`	تخزين مستويات الطلاب ونقاط الـ XP المحصلة وتواريخ التحديث.

Docker Compose (docker-compose.yml) services: postgres: image: postgres:15-alpine container_name: codetree-v2-postgres ports: # إعدادات ["5432:5432"] # واجهة volumes: - ./init-dbs.sql:/docker-entrypoint-initdb.d/init-dbs.sql pgadmin: image: dpage/pgadmin4 ports: ["5050:80"] بصرية لإدارة القواعد عبر المتصفح

7 آلية التشغيل والإيقاف الآلي (Automation & Operations)

نظراً لتعقيد تشغيل 6 خدمات مختلفة اللغات يدوياً، تم إرفاق ملفات دفعية ذكية (Batch Scripts) تدير الترتيب والتوقيتات الحرجة:

سكربت التشغيل: start-all.bat

يقوم بفتح كل خدمة في نافذة Terminal مستقلة ومراقبة الترتيب الحرج لضمان استقرار الاتصالات:

1. **Discovery Service:** تشغيل Eureka والانتظار 15 ثانية لتهيئة السجل.
2. **Gamification Service:** تشغيل خدمة الجافا للربط.
3. **Identity Service:** تشغيل خدمة المستخدمين عبر `dotnet run`.
4. **Content Service:** تشغيل خدمة المحتوى عبر `dotnet run`.
5. **Code Engine:** تفعيل بيئة Python وتشغيل Uvicorn.
6. **API Gateway:** تشغيل البوابة وربط كافة المسارات.

سكربت الإيقاف: stop-all.bat

ينهي العمليات بضغطة واحدة لتحرير المنافذ المحجوزة:

taskkill /F /IM "java.exe" /T taskkill /F /IM "dotnet.exe" /T taskkill /F /IM "python.exe" /T

يضمن إغلاق كافة منافذ العمليات في الذاكرة دون ترك عمليات يتيمة (Zombie Processes).

8 الخلاصة والسمات الهندسية البارزة

- **عزل الأخطاء (Fault Isolation):** تعطل محرك الأكواد أو خدمة التحفيز لن يؤثر إطلاقاً على تصفح المحتوى أو تسجيل الدخول.
- **التدرج والتوسع الأفقي (Horizontal Scalability):** يمكن مضاعفة نسخ خدمة `Code Engine` لمواكبة آلاف الطلاب بالتوازي، ويتكفل خادم Eureka و Gateway بتوزيع الأحمال عليها فوراً.
- **الأمان في تقييم الأكواد:** الفصل الكامل في مجلدات مؤقتة وفرض حد أقصى للوقت (Timeouts) يحمي الخوادم من الاستغلال أو الاستنزاف.